

8-3 · 수집 → Lake → Spark → VectorDB 흐름 자동화 · 실습 1H

임베딩 → VectorDB 적재와 전체 DAG 완성

마지막 태스크를 붙이고, 트리거 한 번으로 끝까지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① embed_and_load 를 붙여 collect → transform → validate → embed_and_load 를 완성한다
- ② 트리거 한 번으로 Qdrant lecture_docs 의 points_count 가 3 이 되는 것을 본다
- ③ 두 번 돌려도 개수가 그대로인 것(멥등)과 @daily 의 Next Run 을 확인한다
- ④ SparkSubmitOperator 제출 형태와 오늘 방식의 DAG 모양이 같음을 설명한다

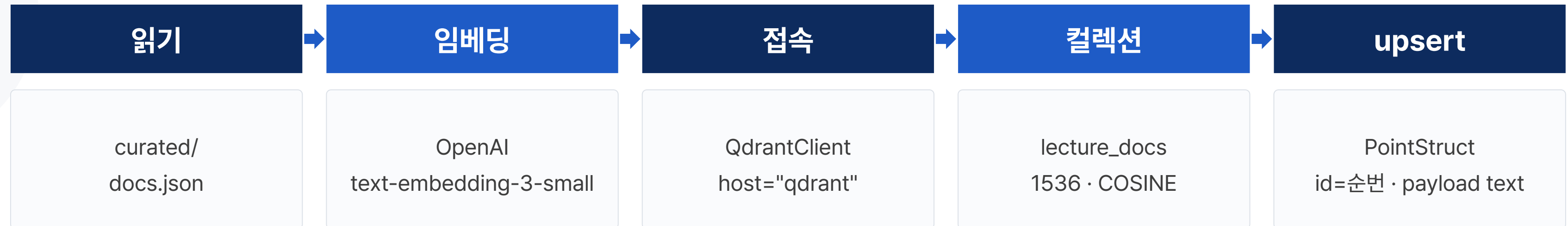
7주 동안 손으로 하던 일이 트리거 한 번으로 끝나는 시간입니다

일곱 단계 — 적재 태스크에서 실행 이력까지

1.5H 에서 초록이 된 세 태스크 뒤에 마지막 하나를 붙입니다

1.2. 적재·배선	3. 전체 실행	4. 두 번 돌리기	5. 스케줄	6.7. Spark·이력
<code>embed_and_load</code> <code>>> t_load</code>	<code>trigger →</code> <code>points_count 3</code>	개수 그대로 = 먹등	<code>@daily</code> Next Run	구조 확인 <code>list-runs</code>

embed_and_load — 함수 안에서 일어나는 일



collection_exists 로 있으면 건너뛰고 없을 때만 **create_collection** — 몇 번 돌려도 같은 결과

import 는 함수 안에 — DAG 파일 최상위에 두면 스케줄러가 파일을 읽을 때마다 임포트가 돌아 전체가 느려진다(8-2 마지막 오류)

배선을 마무리하고 전체를 돌린다

배선	t_collect >> t_transform >> t_validate >> t_load
실행	\$CLI dags trigger rag_pipeline
컬렉션 보기	curl -s http://localhost:6333/collections/lecture_docs python -m json.tool head -20
개수만	curl -s ... /collections/lecture_docs grep -o '"points_count":[0-9]*'
성공 기준	points_count 가 3

Graph 에 collect → transform → validate → embed_and_load 순서도 · Grid 에서 4개가 초록이 된 뒤 curl

4단계

두 번 돌려 먹등을 확인한다

1

지금 개수를 적어 둔다

`grep -o '"points_count":[0-9]*' → 3`

2

한 번 더 **trigger, sleep 60**

`$CLI dags trigger rag_pipeline` — Grid 가 초록이 될 때까지

3

다시 세면 3 그대로

id 를 순번으로 줬기 때문에 같은 자리를 덮어쓴 것

남아 있는 문제

문서 수가 줄면
뒤쪽이 남는다

7-4 에서 본 그 문제가 그대로 있다 — 8-5 최종본에서 다룬다

스케줄을 붙인다 — 사람이 손대지 않아도 매일

이번 주의 목표

손대지 않아도
매일 돈다

- 1** **schedule=None** 을 "**@daily**" 로 바꿔 저장
DAG 인자 한 줄만 바뀐다
- 2** **catchup=False** 는 반드시 유지
없으면 과거 날짜 실행이 쏟아진다
- 3** **sleep 40** 뒤 **dags list | grep rag_pipeline**
스케줄러가 파일을 다시 읽을 시간을 준다
- 4** **웹 UI DAG 목록의 Next Run 열**
다음 실행 시각이 표시되면 이번 주의 목표에 닿은 것

Spark 제출 — 태스크 안의 실행 방식만 다르다

오늘 — PythonOperator

경량 파이썬 변환, 단일 노트북에서 돈다

- `task_id="transform"`
- `python_callable=transform`
- `t_collect >> t_transform >> t_validate >> t_load`

VS

실무 — SparkSubmitOperator

Spark 클러스터에 잡을 제출한다(구조만 확인)

- `application=/opt/airflow/jobs/transform.py(5주차 잡)`
- `conn_id="spark_default"`
- `t_collect >> transform_spark >> t_validate >> t_load` — 배선 동일

embed_and_load 만 죽는 세 갈래 — 미리 겪어 둔다

증상	원인	해결
qdrant 호스트를 찾을 수 없음·연결 거부	따로 docker run 한 Qdrant — 네트워크가 다름	compose 에 서비스 추가 또는 docker network connect
ModuleNotFoundError qdrant_client	.env 추가 뒤 재생성 전 또는 변수 이름 오타	철자 확인 → up -d → python -c "import ..."
임베딩 호출 인증 오류(API key)	.env 에는 있지만 environment: 줄이 없다	yaml 한 줄 → up -d → env grep OPENAI

여유가 있으면 yaml 의 OPENAI_API_KEY 전달 줄을 잠시 지우고 up -d 해 인증 오류를 직접 본 뒤 되돌린다 — 실패를 봐야 이유가 남는다

이 1시간이 끝날 때 확인할 것

- 1 Graph 에 collect → transform → validate → embed_and_load 가 보인다
- 2 트리거 뒤 lecture_docs 의 points_count 가 3 이다
- 3 두 번째 실행 뒤에도 points_count 가 3 그대로다
- 4 @daily 로 바꾼 뒤 Next Run 에 다음 실행 시각이 있다
- 5 dags list-runs · tasks states-for-dag-run 에 태스크별 상태·시간이 남아 있다

run_id 는 따옴표 안에 — < > 로 감싸면 쉘이 리다이렉션으로 읽어 syntax error

8-3 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

오늘 만든 자동화의 방법을 표 하나로 정리하고, 8-4 챗봇이 읽을 lecture_docs 가 채워져 있는지 확인합니다

✓ 하려는 일 ↔ 방법 여덟 줄 요약

✓ 체크리스트로 스스로 점검

✓ 자주 겪는 오류 네 가지

✓ 8-4 Streamlit 챗봇 예고

가져갈 것

points_count 3 인 lecture_docs